PART 3

Framework architecture

The target shape of the finished add-on: four layers with a strict direction of control, a small set of data types, one evaluation pipeline per frame, and a package layout that replaces the single 3,000-line file.

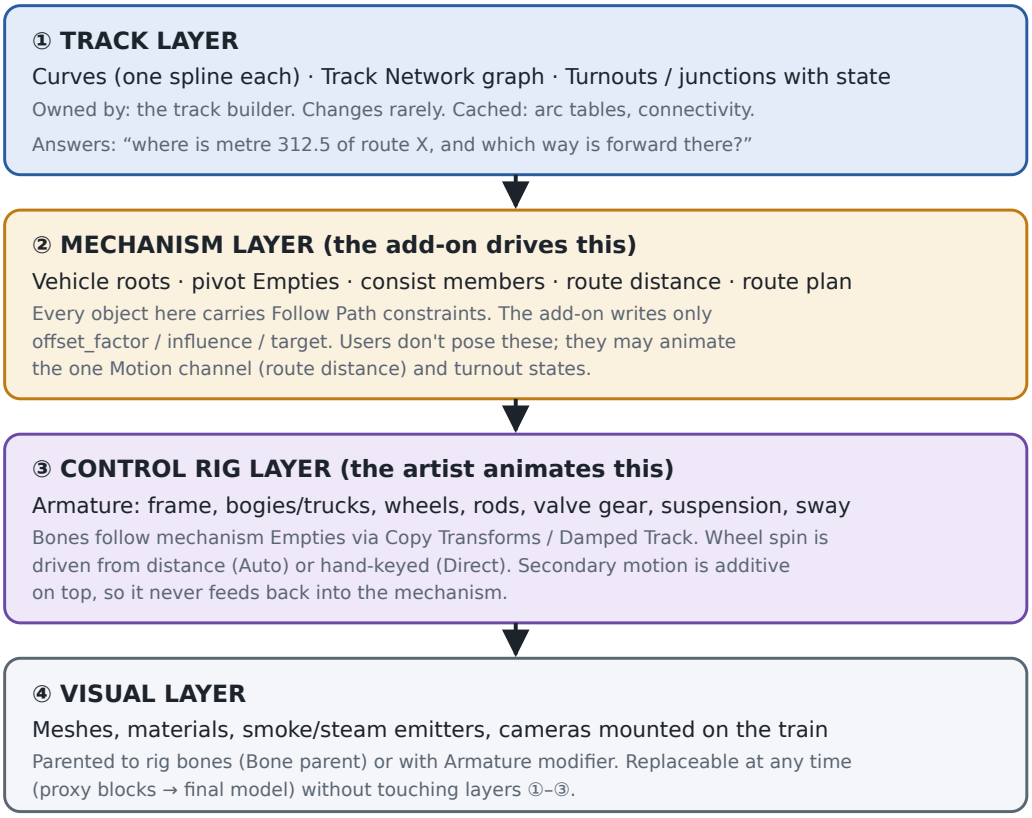
3.1 Design principles
3.2 The four-layer model
3.3 Data model
3.4 Per-frame evaluation pipeline
3.5 Package layout
3.6 Where state lives
3.7 Architecture decisions

3.1 Design principles

Every principle below comes from a decision already made during the sessions. They are written down here so new code can be checked against them.

Principle	In practice
The mesh is never bolted to the mechanism	The add-on moves *roots and pivots*. The rig reads them through constraints, and meshes parent to the rig. Swap a mesh, keep the rig. Swap a rig, keep the mechanism.
Blender does the geometry	Wherever possible the add-on only sets *numbers* (offset_factor, influence, distance) and lets Follow Path, Copy Transforms and Damped Track do the maths. It is inspectable, keyframeable, and banking comes for free.
Distance is the currency	Pivot offsets, coupling gaps, wheel rotation and route history are all real metres. Factors are only ever computed at the last moment, per curve.
Crash-safe live control	No constraint writes from handlers during a modal 3D transform. Live control goes through N-panel widgets (Drag Slider, Drag Pad) or the add-on's own modal operators.
Deterministic playback	The same frame must give the same pose no matter what is selected, whether you scrub forwards or backwards, or whether it's the viewport or a render farm.
No guessing at forks	Curves change only where a decision was recorded or the continuation is unique. Proximity alone never switches a train (the lesson of v8 §4.2).
Everything reversible	Detach keeps constraints, Remove Switch confirms with real counts, and every operator is undoable.

3.2 The four-layer model



Direction of control

Data flows **down only**. Nothing below writes back into a layer above.

What users key

- Route distance (m)
- Turnout states
- Coupled / uncoupled
- Rig controls (wheel mode, slip, sway amount, doors…)

Why it matters

The mine-cart example: the add-on moves the cart along the track; the cart's own rig adds wobble, wheel slip and suspension; the mesh can be swapped any time.

Figure 3.1 The layer stack. The existing add-on implements ① (curves only) and ② (root, pivots, wagons). Layers ③ and ④ are new work (Parts 6, 7, 9, 10).

3.3 Data model

Six types cover the whole system. Blender-side they are `PropertyGroup`s on the scene or on objects, plus a few managed objects. Each carries a stable `uid` string, so renames never break links (audit issue #8).

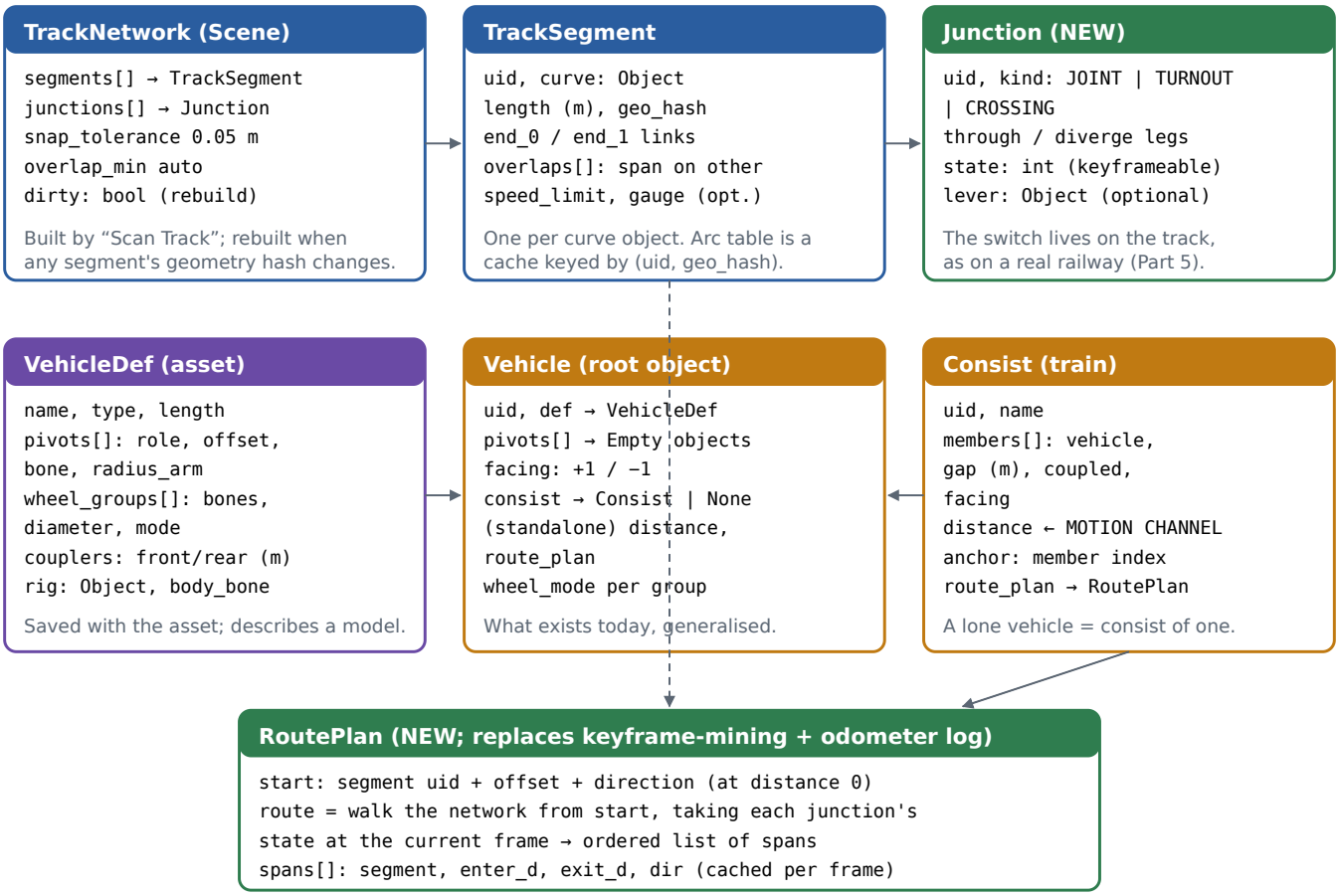
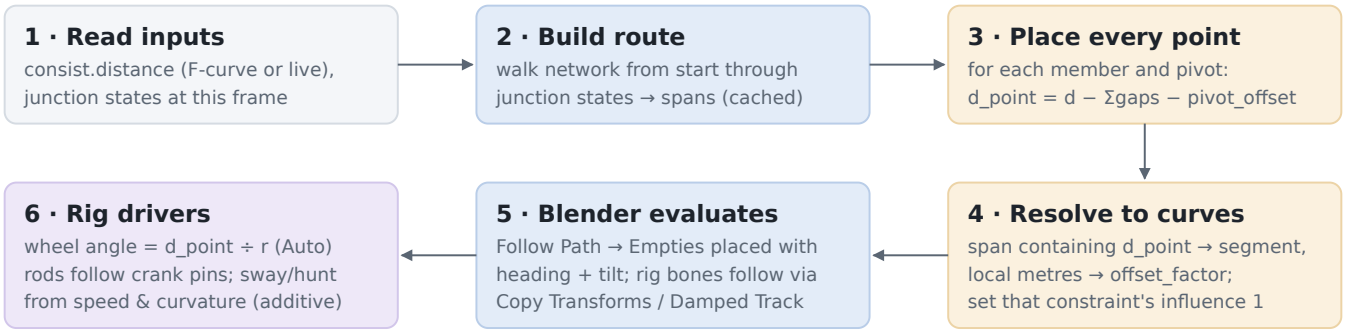


Figure 3.2 Target data model. Green types are new. Amber types exist today in early form (`track_it_vehicle_pivots` , `track_it_consist_wagons` , `track_it_consist_history`).

THE KEY SIMPLIFICATION

Today each vehicle keeps its own history of where it went. In the target model **the train's route is recomputed from two things: where it started, and what the turnouts were set to at each frame**. Anything along the train (root, pivot, wagon, tender bogie) is then “the route at distance *d* − *offset*”. There is no history to go stale, no drift, and no per-wagon fork decisions. Part 5 explains the idea in full; Part 8 applies it to consists.

3.4 Per-frame evaluation pipeline



Where each step runs

Steps 1–4: one Python function, `evaluate_all(scene, frame)`, for **every** consist, called from:

- **frame_change_pre**: playback, scrubbing, and renders (runs before the depsgraph)
- Drag Slider / Drag Pad callbacks: live driving, writes consist.distance first
- operators (Attach, Switch, Couple…), **never** a handler during a modal 3D transform

Steps 5–6: pure Blender (constraints + drivers). No Python, so they evaluate identically in the viewport, in F12 renders and in background (`blender -b`) renders.

Figure 3.3 The target pipeline. The expensive parts (route building, arc tables) are cached and only redone when the inputs change.

RENDER FARMS

Handlers are Python, and some render setups run with scripts disabled (“Auto Run Python Scripts” off) or on machines without the add-on. The pipeline should offer **Bake Train Motion**: write the resolved offset_factor and influence keys for every managed object over a frame range, so the scene plays back with the add-on turned off. See Part 7 §7.8.

3.5 Package layout

Blender loads a folder with `__init__.py` as one add-on (or as an Extension, the Blender 4.2+ format with `blender_manifest.toml`). Splitting by layer keeps each file small and testable:

```
track_it/
  blender_manifest.toml      # extension metadata (replaces bl_info), min Blender 4.4 / 5.x
  __init__.py                # register()/unregister() only
  core/
    arc.py                   # arc tables from the evaluated polyline, cache keyed by (uid,
geo_hash)
    spatial.py               # KD-tree per segment, nearest point/offset queries
    network.py               # TrackNetwork scan: joints, overlaps, turnouts, validation
    route.py                 # RoutePlan walk, distance → (segment, factor, heading)
  mechanism/
    vehicle.py               # root, pivots, facing, constraint management by uid
    consist.py               # members, gaps, coupling, anchor
    evaluate.py              # evaluate_all(): the per-frame pipeline (steps 1–4)
    bake.py                  # Bake Train Motion / Clear Bake
  rig/
    wheels.py                # Auto/Direct/Slip drivers, radius per group
    secondary.py             # sway, hunting, suspension, rail-joint bumps
    bind.py                  # connect rig bones to pivots (Copy Transforms etc.)
  assets/
    vehicle_def.py           # VehicleDef read/write, asset metadata
    wizard.py                # “Make Vehicle” guided setup (Part 9)
  drive/
    slider.py  pad.py        # live controls (Drag Pad = crash-safe mouse drive)
  ui/
    panels.py  lists.py  hud.py  overlays.py
  ops/                       # thin operators that call into the modules above
  handlers.py                # frame_change_pre, load_post, undo_post (rebuild caches)
  tests/                     # headless tests: blender -b -P tests/run.py
```

3.6 Where state lives

State	Stored in	Notes
Track Network, Junctions	Scene PropertyGroup	Saved in the .blend. Rebuilt on demand, or flagged dirty on geometry change.
Junction state	IntProperty (keyframeable)	Constant-interpolation F-curve, which is the switch history. Visible in the Dope Sheet.
Consist distance	FloatProperty (keyframeable)	The one Motion channel. Ease it in the Graph Editor to get acceleration and braking.
Vehicle / pivot / member data	Object PropertyGroups	As today, plus `uid`. Clear these on duplicate (audit #4).
Follow Path constraints	On managed objects	Outputs of the pipeline. Keyframes on them only appear after a Bake.
Arc tables, KD-trees, route spans	Python cache	Rebuilt on `load_post` / `undo_post` / hash change. Never trusted blindly.
HUD message, live-drag timing	Python module globals	UI-only, safe to lose.

3.7 Architecture decisions

Decision	Recommendation	Trade-off
Keep Follow Path, or position Empties directly?	**Keep Follow Path.** It gives banking, is inspectable and bakeable, and matches Blender's own curve sampling.	One constraint per curve per object, managed by uid (that is already the model).
Motion channel: keyed offset_factor or keyed distance?	**Keyed distance on the Consist** (Part 7). Offsets become outputs.	Existing keyframed scenes need a one-time migration operator.
History: keyframe mining, odometer log, or Route Plan?	**Route Plan**: start point + keyframed junction states.	Replaces a lot of working code, so do it in Phase 3, after the Track Network exists.
Evaluate only the active object, or everything?	**Everything**, every frame, in `frame_change_pre`.	Needs the caching work (arc tables, KD-trees) to stay fast.
Single file or package?	**Package / Extension.**	Installation changes from a .py file to a .zip, one time.
Rig: armature per vehicle, or Empties only?	**Armature per vehicle** (Part 6): bones are what animators expect.	A mine cart can use a tiny 3-bone rig. The system doesn't need a complex rig.